`cbegin` should always return a constant iterator

Document #:	P2278R0
Date:	2021-01-10
Project:	Programming Language C++
Audience:	LEWG

Reply-to: Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 How we got to here

A tale in many acts.

§ 1.1 Prologue: Terminology

23.3.1 [iterator.requirements.general]/5 states:

> Iterators that further meet the requirements of output iterators are called *mutable iterators*. Nonmutable iterators are referred to as *constant iterators*.

This paper uses those terms with those meanings: a mutable iterator is one that is writable to, a constant iterator is one that is not writable to.

§ 1.2 Act I: Introduction of Member `cbegin`

In 2004, C++0x had added `auto` but not yet added the range-based for statement. So there was this problem: how do you write a for loop that is immutable? The goal of the paper was quite clear:

> This paper proposes to improve user access to the `const` versions of C++ container `iterators` and `reverse_iterators`.

and:

> However, when a container traversal is intended for inspection only, it is a generally preferred practice to use a `const_iterator` in order to permit the compiler to diagnose `const`-correctness violations

The solution proposed in [N1674] (and later adopted by way of [N1913]) was to add members `cbegin()` and `cend()` (and `crbegin()` and `crend()`) to all the standard library containers, facilitating this code:

```
for (auto it = v.cbegin(), end = v.cend(); it != end; ++it)  {
    //use *it ...
}
```

`c.cbegin()` was specified in all of these containers to perform `as_const(c).begin()`. Although `std::as_const` itself was not added until much later - it is a C++17 feature, first proposed in [N4380].

§ 1.3 Act II: Rise of Non-Member `cbegin`

C++11 thus added the free functions `std::begin` and `std::end`, and member functions `c.cbegin()` and `c.cend()`. But it did not yet have free functions to fetch constant iterators: those were added in 2013 by way of [LWG2128].

While, `std::begin(c)` always calls `c.begin()` (except for C arrays), `std::cbegin(c)` was not specified to call `c.cbegin()`. Instead it, too, called `std::begin(c)` (not even `c.begin()`):

> Implement `std::cbegin/cend()` by calling `std::begin/end()`. This has numerous advantages:
>
> 1. It automatically works with arrays, which is the whole point of these non-member functions.
> 2. It works with C++98/03-era user containers, written before `cbegin/cend()` members were invented.
> 3. It works with `initializer_list`, which is extremely minimal and lacks `cbegin/cend()` members.
> 4. 22.2.1 [container.requirements.general] guarantees that this is equivalent to calling `cbegin/cend()` members.

There are two important goals here to highlight.

First, the goal is still to provide constant iterators, not just call `begin() const`. The latter is an implementation strategy for the former.

Second, the goal is to avoid boilerplate. An implementation where `std::cbegin(c)` called `c.cbegin()` would require `c.cbegin()` to exist, which, as is clear from the list above, is not the case for a lot of useful types.

As a result, `std::cbegin(c)` is basically specified to be `std::begin(as_const(c))` (although, again, predating `std::as_const`) which is basically `as_const(c).begin()`.

The status quo at this point is that `c.cbegin()`, `as_const(c).begin()`, and `std::cbegin(c)` are all equivalent (where they are all valid) and all yield constant iterators.

§ 1.4 Act III: Climax of the Views

Before 2018, the standard library had two non-owning range types: `std::initializer_list<T>` (since C++11) and `std::string_view` (since C++17). Non-owning ranges are shallow-`const`, but both of these types are *always*-`const` so that distinction was insignificant.

That soon changed. 2018 opened with the addition of `std::span` [P0122R7] and closed with the adoption of Ranges [P0896R4], with a few more views added the subsequent year by way of [P1035R7]. Now, for the first time, the C++ standard library had non-owning ranges that were nevertheless mutable. Ranges itself was but a small part of the range-v3 library, so there is a promise of many more views to come.

These types really throw a wrench in the `cbegin` design: because now `begin() const` does not necessarily yield a constant iterator, whereas this had previously always been the case.

It's important to note that while it had previously always been the case *in the standard library*, that is not true for the broad C++ community. In particular, Boost.Range (which begat range-v3 which begat C++20 Ranges) has for a very long time had a type named `boost::iterator_range<It>` (the predecessor to `std::ranges::subrange<It>`). This is a view, although that term hadn't existed yet, and so it had a `begin() const` member function that just returned an `It`. Which means that `std::cbegin` on an `iterator_range<int*>` gives you an `int*` - a mutable iterator.

Where this discrepancy became most apparently visible was the specification of `std::span` during the ballot resolution (by way of [LWG3320]). For the sake of simplicity, I am going to assume that the iterator types of `span<T>` and `span<T const>` are just `T*` and `T const*`, respectively.

— `span<T>::begin() const`, like all the other views, is shallow `const`, and so returns `T*`.

— `span<T>::cbegin() const`, like the other standard library containers, was provided for convenient access to a constant iterator. This returned `T const*`. Unlike the other standard library containers, this did not simply defer to `begin() const`.

So far so good. But because `std::cbegin(s)` is specified to do `std::begin(as_const(s))`, we end up having different behavior between `s.cbegin()` and `std::cbegin(s)`. This is the first (and, thus far, only) type in the standard library for which this is the case - and while `s.cbegin()` would have yielded a constant iterator, `std::cbegin(s)` does not.

As a result of NB comment resolution, to ship a coherent C++20, `span`'s `cbegin()` and `cend()` members were removed, for consistency.

§ 1.5 Intermezzo: Examining the C++20 Status Quo

This leaves us in a state where:

— for all the standard library containers, `r.cbegin()` and `std::cbegin(r)` are equivalent, both meaning `as_const(r).begin()`, and both yielding a constant iterator. This is likely true for many containers defined outside of the standard library as well.

— for most of the standard library views, `r.cbegin()` does not exist and `std::cbegin(r)` is a valid expression that could yield a mutable iterator (e.g. `std::span<T>`). There are three different kinds of exceptions:

1. `std::string_view::cbegin()` exists and is a constant iterator (since it is `const`-only). `std::initializer_list<T>::cbegin()` does *not* exist, but `std::cbegin(il)` also yields a constant iterator.
2. `std::ranges::single_view<T>` is an owning view and is actually thus deep `const`. While it does not have a `cbegin()` member function, `std::cbegin(v)` nevertheless yields a constant iterator (the proposed `views::maybe` in [P1255R6] would also fit into this category).
3. `std::ranges::filter_view<V, F>` is not actually `const`-iterable at all, so it is neither the case that `filt.cbegin()` exists as a member function nor that `std::cbegin(filt)` (nor `std::ranges::cbegin(filt)`) is well-formed. Many other views fit this category as well (`drop_view` being the most obvious, but `drop`, `reverse`, and `join` may not be either, etc.). Other future views may fit into this category as well (e.g. my proposed improvement to `views::split` in [P2210R0]).

Put differently, the C++20 status quo is that `std::cbegin` on an owning range always provides a constant iterator while `std::cbegin` on a non-owning view could provide a mutable iterator or not compile at all.

The original desire of Walter's paper from more than 15 years ago (which, in 2020 terms, may as well have happened at the last Jupiter/Saturn conjunction) still holds today:

> However, when a container traversal is intended for inspection only, it is a generally preferred practice to use a `const_iterator` in order to permit the compiler to diagnose `const`-correctness violations

How could we add `const`-correctness to views?

§ 1.6 A Non-Solution: Member `cbegin()`

One approach we could take to provide reliable `const`-traversal of unknown ranges is to push the problem onto the ranges:

1. We could say that `std::cbegin(c)` (and `std::ranges::cbegin(c)` as well) first tries to call `c.cbegin()` if that exists and only if it doesn't to fall-back to its present behavior of `std::begin(as_const(c))`.
2. We could then pair such a change with going through the standard library and ensuring that all views have a member `cbegin() const` that yields a constant iterator. Even the ones like `std::initializer_list<T>` that don't currently have such a member?

Such a design would ensure that for all standard library ranges, `r.cbegin()` and `std::cbegin(r)` are equivalent and yield a constant iterator. Except for `filter_view`, for which `std::cbegin(filt)` would continue to not compile as it takes a `C const&`.

What does this do for all the views outside of the standard library? It does nothing. `std::cbegin(v)` on such views would continue to yield a mutable iterator, as it does today with `boost::iterator_range`. That, in of itself, makes this change somewhat unsatisfactory.

But what would it actually mean to add a member `cbegin() const` to every view type? What would such a member function do? What it *should* do is the exact same thing for every view — the same exact same thing that all views external to the standard library would have to do in order to opt in to `const`-traversal-on-demand.

But if every type needs to do the same thing, that's an algorithm. The standard library should provide it once rather than having every view re-implement it. Or, more likely, have every view delegate to the algorithm and just have boilerplate member function implementations. A substantial amount of view implementations are already boilerplate, we do not need more.

§ 2 Act IV: `std::const_iterator`

The problem we actually have is this: given an iterator, how do I create an iterator that is identical in all respects except for top-level mutability? This is, ultimately, the problem that from the very beginning `vector<T>::const_iterator` is intending to solve. It is a `vector<T>::iterator` in all respects (it's contiguous, its value type is `T`, it would have the same bounds coming from the same container) except that dereferencing such an iterator would give a `T const&` instead of a `T&`.

§ 2.1 A Reverse Digression

We're already used to the fact that some iterators are generic wrappers over other iterators. `vector<T>` is already specified as:

```
namespace std {
  template<class T, class Allocator = allocator<T>>
  class vector {
  public:
    // types

    using iterator               = implementation-defined; // see [container.requirements]
    using const_iterator         = implementation-defined; // see [container.requirements]
    using reverse_iterator       = std::reverse_iterator<iterator>;
    using const_reverse_iterator = std::reverse_iterator<const_iterator>;
```

Nobody is especially surprised by the fact that every container isn't manually implementing its own bespoke reverse iterators. `std::reverse_iterator<It>` does the job. Yet, `std::rbegin(c)` always calls `c.rbegin()` (except for arrays). Even though we've had this perfectly generic solution for a long time, if you wanted your container to support reverse-iteration, you just had to write these boilerplate `rbegin()`/`rend()` member functions that wrapped your iterators.

Ranges improved this situation. `std::ranges::rbegin(E)` is a much more complicated algorithm that takes many steps (see 24.3.6 [range.access.rbegin] for complete description), but a key aspect of the design there is that if `std::ranges::begin(E)` and `std::ranges::end(E)` give you `bidirectional_iterators`, then `std::ranges::rbegin(E)` itself does the wrapping and provides you `make_reverse_iterator(ranges::end(E))`. No more pushing work onto the containers. That means that it works even in this case:

```
struct simple_span {
    int* begin() const;
    int* end() const;
};

void algo(simple_span ss) {
    auto rit = std::ranges::rbegin(ss); // ok
    auto rend = std::ranges::rend(ss);  // ok
    // ...
}
```

`std::rbegin` would've failed in this case, because we don't have the boilerplate necessary to make it work. But instead of pushing that boilerplate onto `simple_span`, we consigned it into `std::ranges::rbegin`. A much better solution.

§ 2.2 Const Is No Different

A generic `reverse_iterator<It>` is not that complicated. We're basically inverting operations. But the crux of the iterator remains the same: `*it` passes through to its underlying iterator.

A generic `const_iterator<It>` at first seems much less complicated. *Every* operation is passthrough, except for one. We are *only* modifying the behavior of the dereference operator. Yet, doing the right thing for dereference is decidedly non-trivial. Let's go through some cases. We're going to look at both the value type and reference type of several ranges and say what we want the desired `const_iterator<iterator_t<R>>` to dereference into:

	`range_value_t<R>`	`range_reference_t<R>`	desired result type
`vector<int>`	`int`	`int&`	`int const&`
`vector<int> const`	`int`	`int const&`	`int const&`
`array<int const, N>`	`int`	`int const&`	`int const&`
a range of prvalue `int`	`int`	`int`	`int`
`vector<bool> const`	`bool`	`bool`	`bool`
`vector<bool>`	`bool`	`vector<bool>::reference`	`bool`
zipping a `vector<T>` and `vector<U>`	`tuple<T, U>`	`tuple<T&, U&>`	`tuple<T const&, U const&>`

This table points out a few things:

— A first thought might be that we need to return a `range_value_t<R> const&`, but while that works in some cases, it would lead to every element dangling in other cases.

— Sometimes, `It` is already a constant iterator, so we would want to actively avoid wrapping in such a case.

— The last couple rows are hard.

Thankfully, this is a solved problem. The `views::const_` in range-v3 has for many years used a formula that works for all of these cases. In C++20 Ranges terms, I would spell it this way:

```
template <std::input_iterator It>
using const_ref_for = std::common_reference_t<
    std::iter_value_t<It> const&&,
    std::iter_reference_t<It>>;
```

This does not yield the correct result for the last row in my table at the moment, but if we make the changes to `std::tuple` prescribed in [P2214R0], then it would.

Avoiding unnecessary wrapping can be achieved through a factory function that checks to see if such wrapping would change type:

```
// a type is a constant iterator if its an iterator whose reference type is
// the same as the type that const_ref_for would pick for it
template <typename It>
concept constant-iterator = std::input_iterator<It>
```

```
                        && std::same_as<const_ref_for<It>, std::iter_reference_t<It>>;

// a type is a constant range if it is a range whose iterator is a constant iterator
template <typename R>
concept constant-range = std::ranges::range<R> && constant-
        iterator<std::ranges::iterator_t<R>>;

template <std::input_iterator It>
constexpr auto make_const_iterator(It it) {
    if constexpr (constant-iterator<It>) {
        // already a constant iterator
        return it;
    } else {
        return basic_const_iterator<It>(it);
    }
}

template <std::input_iterator It>
using const_iterator = decltype(make_const_iterator(std::declval<It>()));
```

Unfortunately we have a lot of names here:

— `const_iterator<I>` is an alias template that gives you a constant iterator version of `I`. If `I` is already a constant iterator, then `const_iterator<I>` is `I`.

— `make_const_iterator<I>(i)` is a factory function template that takes an input iterator and produces a `const_iterator<I>`. Likewise, if `I` is already a constant iterator, then this function returns `i`.

— `basic_const_iterator<I>` is an implementation detail of the library to satisfy the requirements of the above to ensure that we get a constant iterator.

It's important to have `const_iterator<int const*>` produce the type `int const*`. If we just had a single class template `const_iterator` (that itself was the implementation for a constant iterator), then it could lead to subtle misuse:

```
template <typename T>
struct span {
    using iterator = /* ... */;
    using const_iterator = std::const_iterator<iterator>;
};
```

`span<T const>::iterator` is already a constant iterator, it doesn't need to be wrapped further. So `span<T const>::const_iterator` should really be the same type. It would be nice if the above were already just correct. Hence, the extra names. Users probably never need to use `std::basic_const_iterator<I>` directly (or, indeed, even `make_const_iterator`).

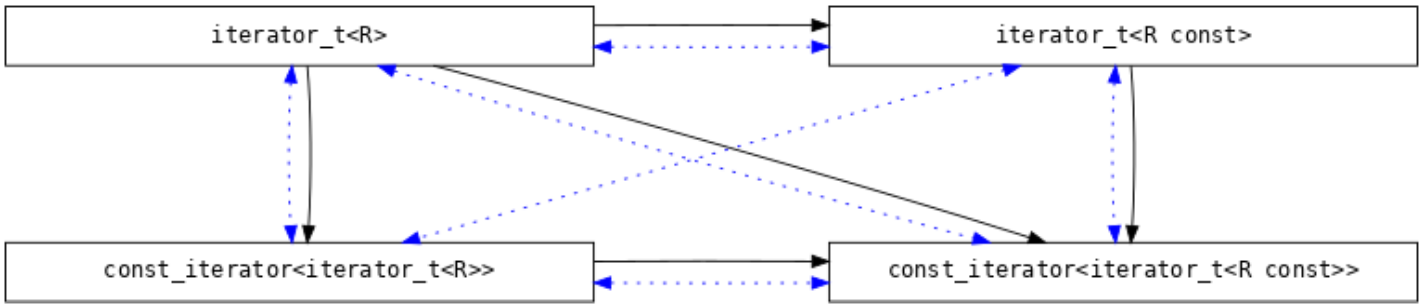
§ 2.3 Implementing `std::basic_const_iterator<I>`

There's a lot of boilerplate in implementing a C++20 iterator. And especially for `basic_const_iterator<I>` where just about every operation is simply a pass-through to the underlying iterator. Only one function ends up having a body consisting of more than a single `return` statement.

Despite that, there's one especially important aspect to implementing a `basic_const_iterator<I>` that adds complexity: conversions and comparisons. We have this expectation that a range's mutable iterator is convertible to its constant iterator. That is, given any type `R` such `range<R>` and `range<R const>` both hold, that

`iterator_t<R>` is convertible to `iterator_t<R const>`. For example, we expect `vector<T>::iterator` to be convertible to `vector<T>::const_iterator`, and likewise for any other container or view.

Adding the concept of a `const_iterator` further complicates matters because now we have the following cross-convertibility and cross-comparability graph:

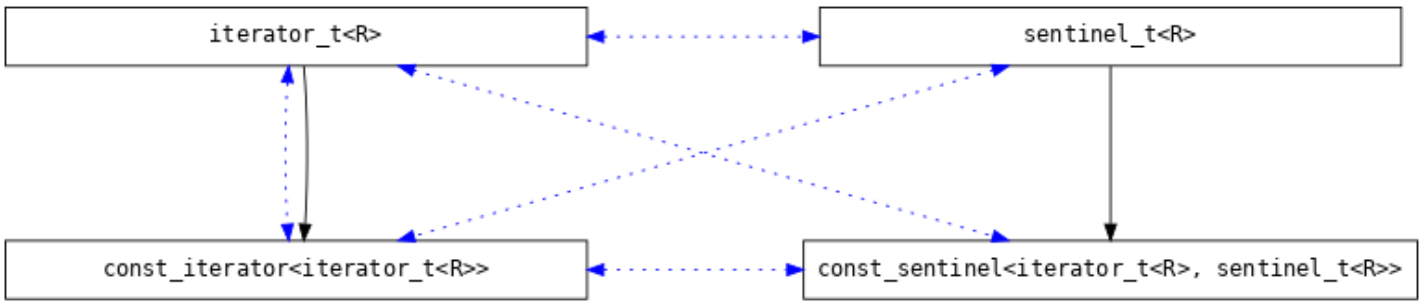


A black arrow from `T` to `U` indicates that `T` needs to be convertible to `U`, while the blue bidirectional dotted arrows between `T` and `U` indicate that `equality_comparable_with<T, U>` holds (i.e. not only that the types can be compared with `==` and `!=` but also that there is a common type between them). That is, every pair of types here needs to model `equality_comparable_with`.

Even though `iterator_t<R const>` and `const_iterator<iterator_t<R>>` are not convertible to each other, they still have a common type that both are convertible to (`const_iterator<iterator_t<R const>>`) which also needs to be reflected.

The implementation of `basic_const_iterator<I>` needs to properly support this graph: which means ensuring the right set of constructors, comparison operators, and even specializations of `common_type`.

The same sort of idea holds for sentinels. How would we wrap sentinels? Imagine a hypothetical `const_sentinel<I, S>`. It would have to have the following behavior:



Because `iterator_t<R const>` would be comparable to `sentinel_t<R>`, it needs to follow that `const_iterator<iterator_t<R>>` needs to be comparable to `sentinel_t<R>` as well. That is, `sentinel_for<const_iterator<I>, S>` needs to hold whenever `sentinel_for<I, S>` holds. This begs the question of if we need a `const_sentinel<I, S>` type at all, given that we need to support comparisons to the unwrapped sentinel anyway. It's a complex interplay of types to get right, and it doesn't seem like a `const_sentinel` type adds value for us at all. Hopefully, we don't find a problem that necessitates wrapping in the future (c.f. [LWG3386]).

The tricky part of the implementation is to avoid constraint recursion in ensuring that wrapped random access iterators are totally ordered and ensuring that wrapped sized sentinels are still sized sentinels. A first attempt at attempting to define an `operator<=>` for `basic_const_iterator<I>` might start with:

```
template <std::input_iterator It>
struct basic_const_iterator {
    template <std::totally_ordered_with<It> Rhs>
        requires std::random_access_iterator<It>
    auto operator<=>(Rhs const& rhs) const;
};
```

But when checking to see if `totally_ordered<basic_const_iterator<int*>>`, we would check to see if we can instantiate `operator<=>`, which requires checking if `totally_ordered_with<basic_const_iterator<int*>, basic_const_iterator<int*>>` (since `Rhs` is our same type), which itself requires checking `totally_ordered<basic_const_iterator<int*>>`. And now we've completed the cycle.

The way I chose to handle this problem is to split the implementation into two functions: a same-type, non-template comparison and a template that is constrained on different types:

```
template <std::input_iterator It>
struct basic_const_iterator {
    auto operator<=>(basic_const_iterator const& rhs) const
        requires std::random_access_iterator<It>;

    template <not-same-as<basic_const_iterator> Rhs>
        requires std::random_access_iterator<It>
             and std::totally_ordered_with<It, Rhs>
    auto operator<=>(Rhs const& rhs) const;
};
```

Other things to note about this implementation:

1. Providing `iterator_concept = contiguous_iterator_tag;` ensures that wrapping a contiguous mutable iterator produces a contiguous constant iterator.
2. Only providing `iterator_category` for `forward_iterators` ensures that we correctly handle C++20 input iterators (more on this later, and see also [P2259R0]).
3. The spelling of the `reference` type for this iterator, described earlier.

```
template <typename It> struct iterator_concept_for { };
template <typename It> requires std::contiguous_iterator<It>
struct iterator_concept_for<It> {
    using iterator_concept = std::contiguous_iterator_tag;
};

template <typename It> struct iterator_category_for { };
template <std::forward_iterator It>
struct iterator_category_for<It> {
    using iterator_category = typename std::iterator_traits<It>::iterator_category;
};

template <std::input_iterator It>
class basic_const_iterator : public iterator_concept_for<It>
                           , public iterator_category_for<It>
{
    It it;

public:
    using value_type = std::iter_value_t<It>;
    using difference_type = std::iter_difference_t<It>;
    using reference = const_ref_for<It>;
```

```
    basic_const_iterator() = default;
    basic_const_iterator(It it) : it(std::move(it)) { }
    template <std::convertible_to<It> U>
    basic_const_iterator(basic_const_iterator<U> c) : it(std::move(c.base())) { }
    basic_const_iterator(std::convertible_to<It> auto&& c) : it(FWD(c)) { }

    auto operator++() -> basic_const_iterator& { ++it; return *this; }
    auto operator++(int) -> basic_const_iterator requires std::forward_iterator<It> { auto cpy
        = *this; ++*this; return cpy; }
    void operator++(int) { ++*this; }

    auto operator--() -> basic_const_iterator& requires std::bidirectional_iterator<It> { --
        it; return *this; }
    auto operator--(int) -> basic_const_iterator requires std::bidirectional_iterator<It> {
        auto cpy = *this; --*this; return cpy; }

    auto operator+(difference_type n) const -> basic_const_iterator requires
        std::random_access_iterator<It> { return basic_const_iterator(it + n); }
    auto operator-(difference_type n) const -> basic_const_iterator requires
        std::random_access_iterator<It> { return basic_const_iterator(it - n); }
    friend auto operator+(difference_type n, basic_const_iterator const& rhs) ->
        basic_const_iterator { return rhs + n; }
    auto operator+=(difference_type n) -> basic_const_iterator& requires
        std::random_access_iterator<It> { it += n; return *this; }
    auto operator-=(difference_type n) -> basic_const_iterator& requires
        std::random_access_iterator<It> { it -= n; return *this; }
    auto operator-(basic_const_iterator const& rhs) const -> difference_type requires
        std::random_access_iterator<It> { return it - rhs.it; }
    auto operator[](difference_type n) const -> reference requires
        std::random_access_iterator<It> { return it[n]; }

    auto operator*() const -> reference { return *it; }
    auto operator->() const -> value_type const* requires std::contiguous_iterator<It> {
        return std::to_address(it); }

    template <std::sentinel_for<It> S>
    auto operator==(S const& s) const -> bool {
        return it == s;
    }

    auto operator<=>(basic_const_iterator const& rhs) const requires
        std::random_access_iterator<It> {
        return it <=> rhs;
    }

    template <not-same-as<basic_const_iterator> Rhs>
        requires std::random_access_iterator<It>
             and std::totally_ordered_with<It, Rhs>
    auto operator<=>(Rhs const& rhs) const {
        if constexpr (std::three_way_comparable_with<It, Rhs>) {
            return it <=> rhs;
        } else if constexpr (std::sized_sentinel_for<Rhs, It>) {
            return (it - rhs) <=> 0;
        } else {
            if (it < rhs) return std::strong_ordering::less;
            if (rhs < it) return std::strong_ordering::greater;
            return std::strong_ordering::equal;
        }
    }

    template <std::sized_sentinel_for<It> S>
```

```
    auto operator-(S const& s) const -> std::iter_difference_t<It> {
        return it - s;
    }

    template <not-same-as<basic_const_iterator> S>
        requires std::sized_sentinel_for<S, It>
    friend auto operator-(S const& s, basic_const_iterator const& rhs) ->
        std::iter_difference_t<It> {
        return s - rhs.it;
    }

    auto base() -> It& { return it; }
    auto base() const -> It const& { return it; }
};

template <typename T, std::common_with<T> U>
struct std::common_type<basic_const_iterator<T>, U> {
    using type = basic_const_iterator<std::common_type_t<T, U>>;
};
template <typename T, std::common_with<T> U>
struct std::common_type<U, basic_const_iterator<T>> {
    using type = basic_const_iterator<std::common_type_t<T, U>>;
};
template <typename T, std::common_with<T> U>
struct std::common_type<basic_const_iterator<T>, basic_const_iterator<U>> {
    using type = basic_const_iterator<std::common_type_t<T, U>>;
};
```

Since the above implementation satisfies the requirements for a `sentinel` where appropriate, we can complete our implementation by providing a `make_const_sentinel` to mirror the `make_const_iterator` shown earlier:

```
template <typename S>
constexpr auto make_const_sentinel(S s) {
    if constexpr (std::input_iterator<S>) {
        // the sentinel here is an iterator in its own right, so we need to (possibly) wrap it
        the same way
        return make_const_iterator(std::move(s));
    } else {
        return s;
    }
}
```

We could take the iterator type as a template parameter to enforce that `S` satisfies `sentinel_for<I>`, but this function is only used as a building block of an algorithm that would already enforce this, so it's probably not necessary.

§ 2.4 Better Algorithms for `std::ranges::cbegin` and `std::ranges::end`

`std::ranges::cbegin` today (24.3.4 [range.access.cbegin]) unconditionally calls `std::ranges::begin`. Similarly, `std::cbegin` today (23.7 [iterator.range]) unconditionally calls `std::begin`. The status quo in the library is that nothing anywhere invokes *member* `cbegin`. The goal is to provide a constant iterator version of `begin()` — we have not had a customization point for this facility in the past and we can achieve this goal without having to add a customization point for the future.

With the above pieces, we implement a `ranges::cbegin` and `ranges::end` to ensure that we get a constant iterator (see full implementation [const-impl], complete with many tests):

```
inline constexpr auto possibly_const = []<std::ranges::range R>(R& r) -> auto& {
    // we only cast to const if it is meaningful to do so
    if constexpr (constant-range<R const> and not constant-range<R>) {
        return const_cast<R const&>(r);
    } else {
        return r;
    }
};

inline constexpr auto cbegin = first_of(
    // 1. non-borrowed rvalue
    delete_if_nonborrowed_rvalue,
    // 2. possibly-wrapped begin of possibly-const r
    [](std::ranges::range auto&& r)
        RETURNS(make_const_iterator(std::ranges::begin(possibly_const(r))))
);

inline constexpr auto cend = first_of(
    // 1. non-borrowed rvalue
    delete_if_nonborrowed_rvalue,
    // 2. possibly-wrapped end of possibly-const r
    [](std::ranges::range auto&& r)
        RETURNS(make_const_sentinel(std::ranges::end(possibly_const(r))))
);
```

Here, `cbegin(r)` and `cend(r)` produce a range that is top-level const over any underlying range, without having to modify any of those underlying ranges to opt in to this behavior. This works for `std::vector<int>` and `std::span<int>` and `boost::iterator_range<int*>` and even views like `std::ranges::filter_view` (`possibly_const` ensures that if get passed a non-`const` `vector<int>`, we treat it as `const` first — which is both valid and necessary — while `filter_view const` isn't a `range` so we cannot treat it as `const` first).

Avoiding a customization point here lets us give an easy answer to the question of whether or not types should provide a member `cbegin` going forward: no, they shouldn't. Users that want a constant iterator can use this facility, which will work for all ranges.

In addition to simply working across all ranges, it has a few other features worth noting:

— For a `vector<int> v`, `cbegin(v)` gives precisely the type `vector<int>::const_iterator` (not `const_iterator<vector<int>::iterator>`).

— For a `span<int> s`, `cbegin(s)` provides a contiguous iterator over `int const&`, not just any kind of iterator.

— For a `transform_view<V, F>`, we *avoid* casting to `const`, since it doesn't do anything for us. But for a `single_view<T>` (which is an owning view and thus deep `const`), we *do* cast to `const`.

Note that here, `ranges::end` already checks that the type returned is a `sentinel` for the type returned by `ranges::begin`. Given that fact, the implementation here already ensures that `sentinel_for<decltype(cbegin(r)), decltype(cend(r))>` holds.

§ 2.5 A `views::const_`

A whole const-view can be implemented on top of these pieces, in the same way that `views::reverse` is implemented on top of `reverse_iterator` (see the full implementation [const-impl]):

```
template <std::ranges::input_range V>
    requires std::ranges::view<V>
class const_view : public std::ranges::view_interface<const_view<V>> {
```

```
    V base = V();
public:
    constexpr const_view() = default;
    constexpr const_view(V base) : base(std::move(base)) { }

    constexpr auto begin() requires (!simple-view<V>) { return cbegin(base); }
    constexpr auto end() requires (!simple-view<V>) { return cend(base); }
    constexpr auto size() requires std::ranges::sized_range<V> { return
        std::ranges::size(base); }

    constexpr auto begin() const requires std::ranges::range<V const> { return cbegin(base); }
    constexpr auto end() const requires std::ranges::range<V const> { return cend(base); }
    constexpr auto size() const requires std::ranges::sized_range<V const> { return
        std::ranges::size(base); }
};

template <typename V>
inline constexpr bool ::std::ranges::enable_borrowed_range<const_view<V>> =
    std::ranges::enable_borrowed_range<V>;

// libstdc++ specific (hopefully standard version coming soon!)
inline constexpr std::views::__adaptor::_RangeAdaptorClosure const_ =
    []<std::ranges::viewable_range R>(R&& r)
    {
        using U = std::remove_cvref_t<R>;

        if constexpr (constant-range<R>) {
            return std::views::all(r);
        } else if constexpr (constant-range<U const> and not std::ranges::view<U>) {
            return std::views::all(std::as_const(r));
        } else {
            return const_view<std::views::all_t<R>>(std::views::all(r));
        }
    };
```

The three cases here are:

1. `r` is already a constant range, no need to do anything, pass it through. Examples are `std::span<T const>` or `std::vector<T> const` or `std::set<T>`.
2. `r` is not a constant range but `std::as_const(r)` would be. Rather than do any wrapping ourselves, we defer to `std::as_const`. Example is `std::vector<T>`. We explicitly remove `views` from this set, because `views::all()` would drop the `const` anyway and we may need to preserve it (e.g. `single_view<T>`).
3. `r` is neither a constant range nor can easily be made one, so we have to wrap ourselves. Examples are basically any mutable view.

To me, being able to provide a `views::const_` is the ultimate solution for this problem, since it's the one that guarantees correctness even in the presence of a range-based for statement:

```
for (auto const& e : r | views::const_) { ... }
```

Or passing a constant range to an algorithm:

```
dont_touch_me(views::const_(r));
```

As far as naming goes, obviously we can't just call it `views::const`. range-v3 calls it `const_`, but there's a few other good name options here like `views::as_const` or `views::to_const`.

§ 2.6 What About `std::cbegin` and `std::cend`?

The above presents an implementation strategy for `std::ranges::cbegin` and `std::ranges::cend`.

But what about `std::cbegin` and `std::cend`? The problem is, while the former is C++20 technology and so can make use of the C++20 iterator model, `std::cbegin` is C++11 technology and so has to remain fixed with the C++11 iterator model. The biggest difference for these purposes has to do with input iterators.

In C++11, even for an input iterator, this is valid code:

```
auto val = *it++;
```

But in C++20, an input iterator's postfix increment operator need not return a copy of itself. All the ones in the standard library return `void`. This is the safer design, since any use of postfix increment that isn't either ignoring the result or exactly the above expression are simply wrong for input iterators.

Trying to be backwards compatible with pre-C++20 iterators is quite hard (again, see [P2259R0]), and the `basic_const_iterator<It>` implementation provided in this paper would *not* be a valid C++17 input iterator. Additionally, C++20 input iterators are required to be default constructible while C++17 input iterators were not.

On the other hand, is it critically important to have a constant iterator for a C++17 input iterator? You're not going to mutate anything meaningful anyway.

A simple solution could have `std::cbegin(c)` pass through C++17 input iterators unconditionally, and otherwise do `make_const_iterator(as_const(c).begin())` (i.e. the `std::ranges::cbegin` described above) for all iterators that are either C++20 iterators or C++17 forward iterators. This probably addresses the majority of the use-cases with minimal fuss.

Moreover, would we want to extend `std::cbegin` to also handle non-`const`-iterable ranges? This is technically doable:

```
template <typename C>
concept can_begin = requires (C& c) { std::begin(c); }

template <class C>
    requires can_begin<const C>
constexpr auto cbegin(const C& c) {
    // today's overload, but also conditionally wrap
}

template <class C>
    requires (can_begin<C> && !can_begin<const C>)
constexpr auto cbegin(C& c) {
    // fallback for non-const-iterable ranges
}
```

I negated the constraint to prefer the `const C&` overload even for non-const arguments.

But we didn't introduce a C++20 iteration model to then go back and give us more work to keep two models in lock-step. The C++20 one on its own is hard enough, and is a better model. We should just commit to it.

It'd probably just be good enough to do something like:

```
template <class C>
constexpr auto cbegin(C const& c)
```

```
    requires requires { std::begin(c); }
{
    if constexpr (std::forward_iterator<decltype(std::begin(c))>) {
        return make_const_iterator(std::begin(c));
    } else {
        // leave input iterators (or... whatever) alone
        return std::begin(c);
    }
}
```

And similarly for `std::cend`. This isn't entirely without peril: currently we say nothing about the constraints for `std::cbegin(r)`; just that it calls `r.begin()`, *whatever that is*. There isn't a requirement that this ends up giving an iterator and there's no requirement that any of the operations that `std::forward_iterator` would check are SFINAE-friendly. I don't know if we necessarily care about such (mis)uses of `std::cbegin`, but it is worth noting.

§ 2.7 Now Reverse It

Now that we can produce a constant iterator for a range, producing a reversed constant iterator is a matter of composing operations. We don't have to worry about sentinels in this case:

```
inline constexpr auto crbegin = first_of(
    // 1. non-borrowed rvalue
    delete_if_nonborrowed_rvalue,
    // 2. possibly-wrapped reversed begin of possibly-const range
    [](std::ranges::range auto&& r)
        RETURNS(make_const_iterator(std::ranges::rbegin(possibly_const(r))))
);

inline constexpr auto crend = first_of(
    // 1. non-borrowed rvalue
    delete_if_nonborrowed_rvalue,
    // 2. possibly-wrapped reversed end of possibly-const range
    [](std::ranges::range auto&& r)
        RETURNS(make_const_iterator(std::ranges::rend(possibly_const(r))))
);
```

Notably here, `ranges::rbegin` and `ranges::rend` already themselves will try to produce a `std::reverse_iterator` if the underlying range is bidirectional. Similarly, we're ourselves trying to produce a `std::const_iterator`.

The extension to `std::crbegin` and `std::crend` mirrors the similar extension to `std::cbegin` and `std::cend`:

```
template <class C>
constexpr auto crbegin(C const& c)
    requires requires { std::rbegin(c); }
{
    if consetxpr (std::forward_iterator<decltype(std::rbegin(c))>) {
        return make_const_iterator(std::rbegin(c));
    } else {
        // leave input iterators (or... whatever) alone
        return std::rbegin(c);
    }
}
```

Even though in the standard library we define `const_reverse_iterator` as `std::reverse_iterator<const_iterator>` and the algorithm presented here constifies a reverse iterator, it still

produces the same result for all standard library containers because `rbegin() const` returns `const_reverse_iterator`, so we already have a constant iterator just from `rbegin`.

§ 2.8 Customizing `make_const_iterator`

The job of `make_const_iterator(it)` is to ensure that its result is a constant iterator that is as compatible as possible with `it` (ideally, exactly `it` where possible). For a typical iterator, `I`, the best we could do if `I` is a mutable iterator is to return a `const_iterator<I>`. But what if we had more information. Could we do better? *Should* we do better?

Consider `char*`. `char*` is not a constant iterator, and the `make_const_iterator` presented here will yield a `const_iterator<char*>`. But there is a different kind of constant iterator we could return in this case that satisfies all the complicated requirements we've laid out for how a constant iterator should behave, and it's kind of the obvious choice: `char const*`. Returning `const char*` here does the advantage in that we avoid having to do this extra template instantiation, which is certainly not free.

The question is: should `make_const_iterator(char*)` yield a `char const*` instead of a `const_iterator<char*>`?

Let's take a look at a test-case. Here is a contiguous range that is a null-terminated mutable character string:

```
struct zsentinel {
    auto operator==(char* p) const -> bool {
        return *p == '\0';
    }
};

struct zstring {
    char* p;
    auto begin() const -> char* { return p; }
    auto end() const -> zsentinel { return {}; }
};
```

You might think `zentinel` is a strange type (why does it compare to `char*` instead of `char const*` when it clearly does not need mutability), but the important thing is that this is a perfectly valid range. `zsentinel` does model `sentinel_for<char*>`, and `zstring` does model `contiguous_range`. As such, it is important that the facilities in this paper *work*; we need to be able to provide a constant iterator here such that we end up with a constant range.

But here, we would end up in a situation where, given a `zstring z`:

— `cbegin(z)` would return a `char const*`

— `cend(z)` would return a `zsentinel` (`zsentinel` is not an iterator, so we do not wrap it)

But while `zsentinel` models `sentinel_for<char*>` and I can ensure in the implementation of `const_iterator<T>` that `zsentinel` also models `sentinel_for<const_iterator<char*>>`, it is not the case that `zsentinel` models `sentinel_for<char const*>`. So we would not end up with a range at all!

It is possible to work around this problem by adding more complexity.

We could add complexity to `make_const_sentinel`, passing through the underlying iterator to be able to wrap `zsentinel` such that it performs a `const_cast` internally. That is, `cend(z)` would return a type like:

```
struct const_zsentinel {
    zsentinel z;
```

```
    auto operator==(char const* p) const -> bool {
        return z == const_cast<char*>(p);
    }
};
```

Such a `const_cast` would be safe since we know we must have originated from a `char*` to begin with, if the `char const*` we're comparing originated from the `zstring`. But there are other `char const*`s that don't come from a `zstring`, that may not necessarily be safe to compare against, and now we're supporting those. This also doesn't generalize to any other kind of `make_const_iterator` customization: this implementation is specific to `const_cast`, which is only valid for pointers. We would have to add something like a `const_iterator_cast`.

Let's instead consider adding complexity in the other direction, to `make_const_iterator`. We can pass through the underlying sentinel such that we only turn `char*` into `char const*` if `S` satisfies `sentinel_for<char const*>`, and otherwise turn `char*` into `const_iterator<char*>`. We could make this more specific and say that we turn `char*`into `char const*` only if *both* the `iterator` and `sentinel` types are `char*`, but I don't think the generalization that `sentinel_for<char const*>` is sufficient.

This direction can be made to work and I think, overall, has fewer problems with the other direction. That is, something like this:

```
template <typename S, std::input_iterator It>
    requires std::sentinel_for<S, I>
constexpr auto make_const_iterator(It it) {
    if constexpr (ConstantIterator<It>) {
        return it;
    } else if constexpr (std::is_pointer_v<It>
                    and std::sentinel_for<S, std::remove_pointer_t<It> const*>) {
        return static_cast<std::remove_pointer_t<It> const*>(it);
    } else {
        return basic_const_iterator<It>(it);
    }
}
```

But is it worth it?

A benefit might be that for a simple implementation of `span`, we could have this:

```
template <typename T>
struct simple_span {
    auto begin() const -> T*;
    auto end() const -> T*;

    auto cbegin() const -> T const*;
    auto cend() const -> T const*;
};
```

Which out of the box satisfies that `simple_span<int>::cbegin()` and `cbegin(simple_span<int>)` both give you an `int const*`. Whereas, with the design presented up until now, member `cbegin` and `cend` would have to return `const_iterator<T*>` instead.

I'm not sure it's worth it. The simpler design is easier to understand. There's something nice about `make_const_iterator` being able to act on an iterator in a vacuum, and being able to define `cbegin` simply as:

```
make_const_iterator(std::ranges::begin(possibly_const(r)))
```

As opposed to:

```
auto& cr = possibly_const(r);
make_const_iterator<std::ranges::sentinel_t<decltype(cr)>>(std::ranges::begin(cr))
```

and having to extend the alias template to:

```
template <std::input_iterator I, std::sentinel_for<I> S = I>
using const_iterator = decltype(make_const_iterator<S>(std::declval<I>()));
```

which still allows `std::const_iterator<int const*>` to be `int const*`. But I'm not sure it's worth it for *just* the pointer case for *just* being able to define `simple_span`. In this model, `simple_span` wouldn't even need to define member `cbegin`/`cend`, so why would it do so, and provide something different?

I could be convinced otherwise though.

§ 2.9 What does this mean for span<T>?

There has been desire expressed to provide member `cbegin` for `span<T>`. This paper allows a meaningful path to get there:

```
template <typename T>
struct span {
    using iterator = implementation-defined;
    using const_iterator = std::const_iterator<iterator>;
    using reverse_iterator = std::reverse_iterator<iterator>;
    using const_reverse_iterator = std::const_iterator<reverse_iterator>;

    auto begin()   const noexcept -> iterator;
    auto cbegin()  const noexcept -> const_iterator;
    auto rbegin()  const noexcept -> reverse_iterator;
    auto crbegin() const noexcept -> const_reverse_iterator;

    // similar for end
};
```

The above design ensures that given a `span<int> s` (or really, for any `T` that is non-`const`):

1. `std::ranges::begin(s)` and `s.begin()` have the same type
2. `std::ranges::cbegin(s)` and `s.cbegin()` have the same type
3. `std::ranges::rbegin(s)` and `s.rbegin()` have the same type
4. `std::ranges::crbegin(s)` and `s.crbegin()` have the same type

All without having to add new customization points to the standard library. Notably, `span<int>::const_iterator` and `span<int const>::iterator` will not be the same type (which arguably a good thing). But `span<int const>::iterator` and `span<int const>::const_iterator` will be the same type (which is important).

The one tricky part here is `const_reverse_iterator`. For all the containers currently in the standard library, we define `const_reverse_iterator` as `std::reverse_iterator<const_iterator>`, but if we did that here, we'd end up with a mismatch in (4): `crbegin(s)` would return a `const_iterator<reverse_iterator<iterator>>` while `s.crbegin()` would return a `reverse_iterator<const_iterator<iterator>>`. We end up applying the layers in a different order.

But this is fine. None of these member functions are even strictly necessary, and there isn't any need for future shallow-const views to provide them. If we want to duplicate already-existing functionality, we should just make sure that we duplicate it consistently, rather than inconsistently.

§ 3 Act V: A Concluding Proposal

The status quo is that we have an algorithm named `cbegin` whose job is to provide a constant iterator, but it does not always do that, and sometimes it doesn't even provide a mutable iterator. This is an unfortunate situation.

We can resolve this by extending `std::ranges::cbegin` and `std::ranges::cend` to conditionally wrap their provided range's `iterator`/`sentinel` pairs to ensure that the result is a constant iterator, and use these tools to build up a `views::const_` range adapter. This completely solves the problem without any imposed boilerplate per range.

However, `std::cbegin` and `std::cend` are harder to extend. If we changed them at all, we would probably punt on handling C++17 input iterators and non-`const`-iterable ranges. This means that `std::cbegin` and `std::ranges::cbegin` do different things, but `std::rbegin` and `std::ranges::rbegin` *already* do different things. `std::ranges::rbegin` is already a superior `std::rbegin`, so having `std::ranges::cbegin` be a superior `std::cbegin` only follows from that. In other words, `std::cbegin` is constrained to not deviate too much from its current behavior, whereas `std::ranges::cbegin` is new and Can Do Better.

Would it be worth making such a change to `std::cbegin`?

Ultimately, the question is where in the Ranges Plan for C++23 [P2214R0] such an improvement would fit in? That paper is focused exclusively on providing a large amount of new functionality to users. The facility proposed in this paper, while an improvement over the status quo, does not seem more important than any of that paper. I just want us to keep that in mind - I do not consider this problem in the top tier of ranges-related problems that need solving.

§ 4 Epilogue

Thanks to Tim Song for helping me work through the design and implementation details of this paper. Thanks to Peter Dimov and Tomasz Kamiński for insisting on design sanity (even as they insisted on different designs) and providing feedback. Thanks to Eric Niebler for having already solved the problem of how to come up with the right reference type for a `const_iterator<It>` in range-v3.

§ 5 References

[const-impl] Barry Revzin. 2020. Implementing `cbegin`, `cend`, and `const_view`.
https://godbolt.org/z/Px6Gfe

[LWG2128] Dmitry Polukhin. Absence of global functions cbegin/cend.
https://wg21.link/lwg2128

[LWG3320] Poland. span::cbegin/cend methods produce different results than std::[ranges::]cbegin/cend.
https://wg21.link/lwg3320

[LWG3386] Tim Song. elements_view needs its own sentinel type.
https://wg21.link/lwg3386

[N1674] Walter E. Brown. 2004-08-31. A Proposal to Improve const_iterator Use from C++0X Containers.
https://wg21.link/n1674

[N1913] Walter E. Brown. 2005-10-20. A Proposal to Improve const_iterator Use (version 3).
https://wg21.link/n1913

[N4380] ADAM David Alan Martin, Alisdair Meredith. 2015-02-05. Constant View: A proposal for a std::as_const helper function template.
https://wg21.link/n4380

[P0122R7] Neil MacIntosh, Stephan T. Lavavej. 2018-03-16. span: bounds-safe views for sequences of objects.
https://wg21.link/p0122r7

[P0896R4] Eric Niebler, Casey Carter, Christopher Di Bella. 2018-11-09. The One Ranges Proposal.
https://wg21.link/p0896r4

[P1035R7] Christopher Di Bella, Casey Carter, Corentin Jabot. 2019-08-02. Input Range Adaptors.
https://wg21.link/p1035r7

[P1255R6] Steve Downey. 2020-04-05. A view of 0 or 1 elements: views::maybe.
https://wg21.link/p1255r6

[P2210R0] Barry Revzin. 2020-08-14. Superior String Splitting.
https://wg21.link/p2210r0

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges.
https://wg21.link/p2214r0

[P2259R0] Tim Song. 2020-11-20. Repairing input range adaptors and counted_iterator.
https://wg21.link/p2259r0